

COMPSCI 532: Systems for Data Science
Homework 4 - Flink - Turn-In Report

Submitted by:
Girirajan Soudnarrajan
Rahiq Majeed

GitHub Link: <https://github.com/gsoundarraja/frauddetection>

Deliverables:

Question 1: *How do instantiations of the Alert class reach the log4j logging mechanism? Please list the classes that the Alerts are passed through. A place to start is by looking at which files import the Alert class and which files import the log4j logging library (org.slf4j).*

- FraudDetector detects suspicious transactions and emits new alert objects using `out.collectg(alert)`
- FraudDetectorJob connects the alert stream to a sink named AlertSink
- AlertSink logs alerts via the log4j2
- FraudDetector → FraudDetectionJob → AlertSink → Log4j

Question 2: *Why do you think that the next sequence of transactions should not generate an alert?*

Transaction 1: (Account 1, Timestamp 1.005 s, \$2, Zip 01003)

Transaction 2: (Account 2, Timestamp 20 s, \$501, Zip 02115)

Transaction 3: (Account 1, Timestamp 55 s, \$1000, Zip 78712)

Answer: This transaction sequence should not generate an alert because the small and large transactions occur in different zip codes.

- Transaction 1 is a small transaction ($\$2 < \10) for Account 1 at Zip Code 01003
- Transaction 3 is a large transaction ($\$1000 \geq \500) for Account 1 at Zip Code 78712

Our DetailedFraudDetector only triggers an alert when both the small and large transactions occur in the same zip code. Since 01003 is not equal to 78712, the system assumes this is a legitimate scenario where the account holder has traveled to a different location. This prevents false positives that would occur if we only checked transaction amounts without considering location. The different zip codes suggest normal travel behavior rather than fraudulent activity where a stolen card is used in the same location.

Question 3: *In this homework we focused on a credit fraud use case, can you think of any other real life situations where a stream processing approach might be useful? Why do you think so?*

Answer: Stream processing is useful in many real-world scenarios that require real-time analysis, some we can think of are:

1. **Network Security/Intrusion Detection:** Monitor network traffic in real-time to detect unusual patterns, DDoS attacks, or unauthorized access attempts. Immediate detection is critical to prevent data breaches.
2. **Stock Market Trading:** Analyze market data streams to detect price anomalies, execute algorithmic trades, or identify market manipulation. Decisions must be made in milliseconds as market conditions change rapidly.
3. **IoT Sensor Monitoring:** Process data from thousands of sensors (temperature, pressure, vibration) in manufacturing plants or smart buildings to detect equipment failures before they occur, enabling predictive maintenance.
4. **Real-time Recommendation Systems:** Services like Netflix or Spotify can analyze user behavior streams (what users are watching/listening to) to provide immediate personalized recommendations, improving user engagement.

Stream processing is valuable in these scenarios because they all require low-latency decision making based on continuously generated data, where delayed responses would significantly reduce the system's effectiveness or value.